Frequent Itemset Mining: Framework Design and Performance Analysis of Apriori, PCY, and SON Algorithms

Algorithms for Massive Data – End-Of-Course Project

Leila Cielok

Data Science for Economics (Master's Degree)
II Year

June 29, 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

Frequent Itemset Mining (FIM), together with Association Rule Mining (ARM), represent fundamental tasks in data mining and exploratory data analysis. These techniques, originally formalized for market-basket analysis to discover relationships between products purchased together by customers, are now widely used in diverse fields, including plagiarism detection and biological data analysis. In this project, this flexibility is exploited by applying them directly to cinematic collaboration networks.

From a computational perspective, finding frequent itemsets presents a major challenge due to the combinatorial explosion of candidate generation. If a dataset contains d unique items, the total search space of potential itemsets spans an unmanageable 2^d combinations. For this reason, standard counting approaches risk to cause bottleneck due to CPU or memory limitations.

To address these scalability issues, various algorithmic paradigms have been developed to avoid counting unncecessary itemsets. This project evaluates and contrasts three algorithms for finding frequent itemsets within a unified Python framework, starting from the A-Priori methodology, which systematically prunes the search space by leveraging the monotonicity principle (an itemset can only be frequent if all of its subsets are also frequent). Next, a variant of the basic A-Priori is introduced through the implementation of the PCY (Park-Chen-Yu) algorithm, which optimizes the A-Priori by applying a localized hash function to the candidate pairs and creating a memory-efficient bitmap during the first pass to reduce the candidate pair search space in the second pass. Finally, the partitioning-based SON (Savasere, Omiecinski, and Navathe) methodology is presented, which divides the dataset into independent chunks to find local frequent itemsets. A second global pass is then executed to validate these candidates, making this framework specifically suitable for large, distributed, or out-of-core datasets.

By evaluating these implementations on a structured dataset under varying support thresholds and hyperparameter settings, this report characterizes the empirical performance boundaries and efficiency profiles of each approach.

2 Dataset and Pre-Processing

The empirical analysis is conducted on the IMDb Movies Dataset, which comprises the 1,000 top-rated movies and television series compiled from the official IMDb platform.[1] Each record (movie) represents a basket, and the individual items within each basket correspond to the cast members (`"Star1", "Star2", "Star3", "Star4"`) credited in that film.

Each movie record is parsed by extracting the credited cast members and aggregating them into a native Python `set`. This computational choice guarantees that any duplicate actor entries within the same row are instantly collapsed, ensuring transaction integrity. Furthermore, representing baskets as native Python `set` structures drastically optimizes performance. When checking if a candidate pair or triple belongs to a movie, Python's internal hash-table lookup allows for instant verification in constant time ($\mathcal{O}(1)$), completely avoiding slow, item-by-item linear scans.

To eliminate heavy string comparison workloads, the pipeline maps items into integers using a simple indexing schema. While the initial mapping is computed during dataset loading, the remaining execution indexes are populated dynamically:

1. Global translation table (`names2int`): a raw lookup table mapping every unique string actor identifier to a distinct sequential integer index.

2. Frequent singletons array (`frequent_items_table`): a dense re-indexing vector derived from the filtering of size-1 itemsets. To minimize memory footprint and optimize candidate

[1] The dataset is publicly available on Kaggle: `https://www.kaggle.com/datasets/harshitshankhdhar/imdb-dataset-of-top-1000-movies-and-tv-shows`

tracking, each globally indexed item i that satisfies the minimum support threshold is re-mapped to a new identifier starting from 1; items failing the threshold are explicitly mapped to 0 to immediately prune them from the search space.

3. Reverse translation table (`newint2name`): a dynamic map compiled by combining the properties of the previous tables. It is used exclusively at the conclusion of execution loops to re-translate the final processed numerical itemset pairs or triples back to their original readable text strings.

3 Algorithms and Implementations

To ensure a rigorous and consistent comparative analysis, all three algorithms are evaluated under a unified experimental configuration, with a minimum support threshold mathematically tied to the dataset size to ensure scalability. Given the dataset size of $N = 1,000$ records, a minimum support ratio of 0.5% is established, which translates to a strict minimum support count $\lfloor 1000 \times 0.005 \rfloor = 5$ Therefore, any item or itemset appearing fewer than 5 times across the corpus is dynamically pruned.

3.1 Apriori Implementation

The standard `apriori` implementation proceeds level-by-level. During the first Pass, the algorithm scans the dataset to count individual items and identify frequent singletons. In Pass 2, candidate pairs are generated from the frequent singletons by computing combinations over their dense identifiers, and the dataset is scanned again to count and retain only frequent pairs.

To transition to size-3 itemsets, the `find_frequent_triples` subroutine performs a further pass over the baskets. For each basket, it first isolates its frequent items, and then generates candidate trios dynamically via local combinations. To apply the monotonicity principle efficiently, the previously validated frequent pairs are cast into a flat Python `set`, which allows for immediate lookups to guarantee that a candidate trio $\{A, B, C\}$ is tracked and incremented only if all of its constituent sub-pairs ($\{A, B\}$, $\{B, C\}$, and $\{A, C\}$) are confirmed to be frequent.

3.2 PCY (Park-Chen-Yu) Implementation

The `pcy` variant improves pair counting by using an array of hash buckets (`bucket_counts`) with size `N_BUCKETS = 500`. This value was selected empirically through a hyperparameter experiment, in order to obtain a good trade-off between memory usage and the ability to filter candidate pairs (Section 4).

During the first pass, while single items are being counted, the algorithm simultaneously processes all pairs occurring in each basket by hashing them into the bucket array. Formally, for every pair of actors extracted from a basket, their names are resolved to their respective integer identifiers i and j and sorted such that $i < j$ to guarantee order-invariance. The destination bucket index is then computed via the following hash function:

$$\texttt{bucket} = \big((i \cdot 31) + j\big) \pmod{\texttt{N\_BUCKETS}} \tag{1}$$

Subsequently, a bitmap is derived by verifying whether each bucket's accumulated counter meets the minimum support threshold. In the second pass, candidate pairs are explicitly evaluated only if their hashed destination maps to a `True` bit state, confirming that their allocated bucket is frequent. This mechanism drastically reduces the search space for candidate pairs, which in turn optimizes the downstream generation of size-3 itemsets.

While this hashing strategy significantly reduces memory overhead during candidate generation, it introduces potential false positives due to hash collisions (i.e., an infrequent pair landing in a frequent bucket). Consequently, the second pass concludes with a deterministic

evaluation loop over the remaining candidate pairs, filtering out collisions by computing their exact support against the dataset to ensure total frequency accuracy.

3.3 SON Implementation

The SON algorithm eliminates both false negatives and false positives by leveraging a fundamental monotonic property: if an itemset is globally frequent across the entire dataset, it must be locally frequent in at least one of its partitions. Accordingly, by partitioning the dataset, aggregating local frequent itemsets, and validating them globally, SON guarantees the identification of all true frequent patterns while completely avoiding memory allocation crashes.

To prevent memory bottlenecks during this distributed phase, the dataset is split into `N_CHUNKS = 3` partitions. This value was selected empirically through the experiments described in Section 4, as it provided an effective trade-off between local memory consumption and processing overhead.

To account for the framework's sampling architecture, the local support threshold is dynamically scaled down based on the chunk size. Formally, for a partition representing a fraction $p \approx 0.333$ of the complete dataset, the local threshold is adjusted using the following ceiling function to ensure a strict integer limit:

$$\texttt{local\_support} = \left\lceil \frac{\texttt{MIN\_SUPPORT\_COUNT} \cdot \texttt{len(chunk)}}{\texttt{len(baskets)}} \right\rceil \tag{2}$$

Given the global threshold of 5, this formula yields a localized support of $\lceil 5 \times 0.333 \rceil = \lceil 1.665 \rceil = 2$.

The execution then proceeds through a two-pass Map-Reduce architecture:

- Pass 1 (Map Phase): The algorithm executes the standard `apriori` pipeline on each data chunk independently, and the local frequent patterns discovered within each partition are subsequently consolidated into a single global candidate set.
- Pass 2 (Reduce Phase): Because local thresholds are scaled down, some itemsets may pass locally purely due to local fluctuations, acting as temporary false positives. Consequently, this second pass performs a global streaming count over the full dataset, verifying the exact global support of the candidates and filtering out any invalid entries.

4 Hyperparameter Optimization Framework

To observe execution dependencies on internal constants, hyperparameter adjustments are applied across standalone loops for both the PCY and SON frameworks. Specifically, the PCY hashing resolution is evaluated over a range of hash table sizes, $N_{\text{buckets}} \in \{20, 100, 500, 1000, 5000\}$, to observe collision penalties and structural pruning behaviors. In a separate analysis, the SON chunking granularity is tracked across distinct data partition sizes, $N_{\text{chunks}} \in \{3, 6, 10, 20\}$, to examine the operational trade-offs of localized parallel scaling.

Figure 1 illustrates the execution response curves under these isolated hyperparameter permutations, showing distinct structural insights into the optimization limits of both frameworks.

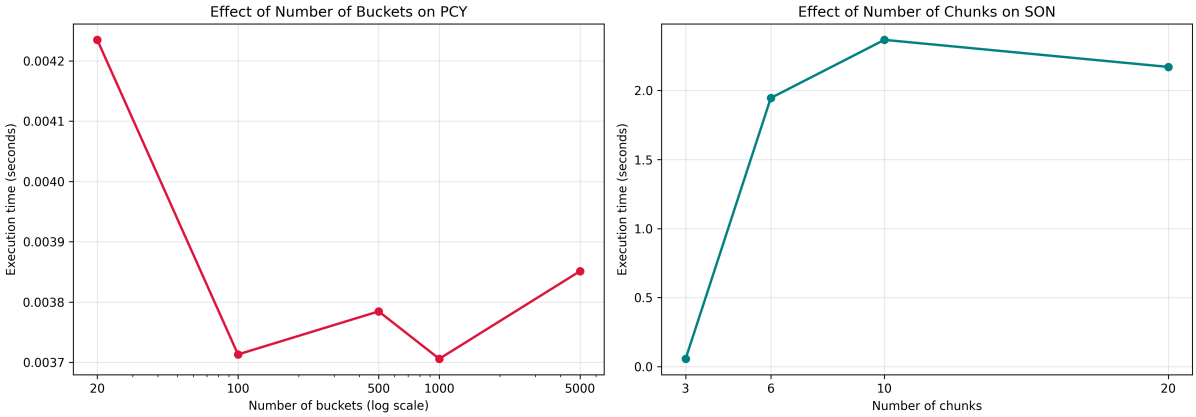

Figure 1: Empirical evaluation of hyperparameter scaling profiles: PCY runtime variation across hash bucket counts (left) and SON framework behavior under distinct data partition granularities (right).

In the left-hand plot, the execution profile of the PCY algorithm highlights the direct impact of collision penalties when the hashing resolution is insufficient. At $N_{\text{buckets}} = 20$, the highly restricted bucket space causes severe hash collisions, driving up candidate verification overhead and generating the worst-case runtime (≈ 0.00424 seconds). As the resolution scales to 100 and up to 1000 buckets, the runtime drops significantly and stabilizes within a narrow, sub-millisecond margin (0.0037 to 0.00385 seconds) due to the decrease in bucket saturation. Within this optimal zone, the minor fluctuation at 500 buckets represents a transient hardware caching or CPU scheduling artifact rather than structural inefficiency. Therefore, `N_BUCKETS` $= 500$ was chosen as a robust operational choice to minimize collisions while remaining computationally lightweight.

As for the SON algorithm, the partition-tuning graph on the right uncovers a severe, binary threshold driven by candidate congestion. At $N_{\text{chunks}} = 3$, the algorithm performs optimally (≈ 0.05 seconds) as the local support constraint evaluates to $\lceil 5 \times 0.333 \rceil = 2$, which is high enough to heavily suppress false positives. Analytically, this defense holds even at 4 partitions since the ceiling function maintains the threshold at $\lceil 5 \times 0.25 \rceil = 2$.

However, as the partitioning increases to 6 or more chunks, the localized support requirement drops below 1.0 and collapses to its minimal absolute threshold (`local_support` $= 1$). This threshold relaxation allows an overwhelming volume of candidate itemsets to clear the first pass simultaneously, as any pattern appearing even once within a single partition is promoted to the global candidate pool. This massively burdens the Reduce phase, causing execution times to skyrocket and plateau heavily between 1.95 and 2.35 seconds.

5 Workload Complexity Scaling via Support Manipulation

Because the dataset size is fixed at only 1,000 rows by external constraints, scalability testing is conducted by manipulating the workload complexity, specifically by reducing the support ratio across four test tiers: `support_ratios` $= \{0.05, 0.01, 0.005, 0.002\}$. Lowering the support ratio broadens the search criteria, resulting in an exponential increase in candidate items, pairs, and triples. This tests how each algorithm copes under sudden, combinatorial stress.

The empirical properties observed during execution are detailed in Table 1 and visualized in Figure 2.

Table 1: Performance and Itemset Generation Metrics Across Scaling Workloads

Ratio	Count	A-Priori (s)	PCY (s)	SON (s)	Singletons	Pairs	Triples	Total
0.050	50	0.0074	0.0084	0.0086	0	0	0	0
0.010	10	0.0109	0.0123	0.0251	9	0	0	9
0.005	5	0.0042	0.0058	0.0587	79	3	1	83
0.002	2	0.0064	0.0092	1.8283	641	121	25	787

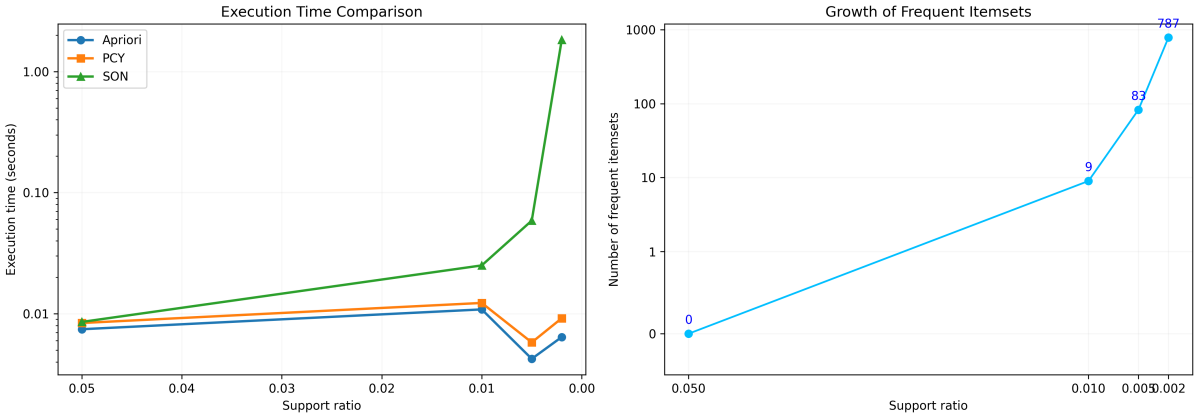

Figure 2: Empirical evaluation of framework scaling: execution runtime trends across Apriori, PCY, and SON pipelines (left) and the corresponding exponential growth in total discovered frequent itemsets (right) as the support threshold relaxes.

When operating at a high support ratio ($R = 0.050$), the threshold is too restrictive for this specific 1,000-row IMDb dataset slice, yielding zero frequent items across all cardinalities. As the support ratio drops to 0.010, a modest baseline of 9 frequent singletons emerges, yet no pairs or triples manage to cross the absolute support threshold of 10 instances. At these high-to-moderate thresholds, all three algorithms perform optimally with minimal overhead, completing their execution in less than 0.03 seconds.

A sharp structural transition in the number of frequent itemsets is observed when shifting the constraint down to $R = 0.002$. At the chosen baseline of $R = 0.005$ (absolute count of 5), the dataset contains a manageable core of 79 frequent singletons, 3 frequent pairs, and 1 frequent triple. However, scaling the ratio down to 0.002 (absolute count of 2) triggers an explosive proliferation of frequent structures, with singletons increasing by a factor of over 8, frequent pairs by a factor of 40, and with frequent triples scaling to 25.

This combinatorial explosion has different impacts on the performance profiles of the three implementations. A-Priori and PCY exhibit exceptional resilience across the entire scaling spectrum, both completing the execution process in less than 0.01 seconds. Conversely, the SON Framework suffers a severe performance penalty at the lowest threshold; while its execution time is highly competitive down to $R = 0.005$ (≈ 0.059 seconds), it surges exponentially to ≈ 1.83 seconds at $R = 0.002$, rendering the partition-based approach roughly 285 times slower than the baseline A-Priori.

This behavioral divergence highlights a fundamental architectural misfit between partition-based chunking and small, low-threshold in-memory workloads, the structural mechanics of which are evaluated in the final discussion.

6 Experimental Results

While the previous analyses evaluate computational efficiency, this section details the specific frequent patterns discovered within the IMDb dataset slice.

At the baseline support threshold (count = 5), the mining process isolates 79 frequent singletons, which correspond to highly prolific actors who frequently appear in top-rated cinematic productions, led by Robert De Niro with a support count of 17, Tom Hanks with 14, and Al Pacino with 13.

Regarding frequent pairs of size 2, only three itemsets manage to cross the minimum absolute support count of 5 instances. Interestingly, these pairs are strictly restricted to the core cast members of the Harry Potter cinematic franchise, specifically capturing the recurring co-occurrences of Daniel Radcliffe and Rupert Grint with a support count of 6, Daniel Radcliffe and Emma Watson with a support count of 5, and Emma Watson and Rupert Grint with a support count of 5.

Moving to frequent triples of size 3, the system isolates a single, definitive pattern containing the complete primary cast of the franchise, namely the trio composed of Emma Watson, Daniel Radcliffe, and Rupert Grint, which possesses a stable support count of 5. Naturally, this specific triple represents the only logical outcome expected from the mining process: according to the downward-closure property of frequent itemsets, a larger structure cannot be frequent unless all of its smaller subsets are also frequent. Because Radcliffe, Grint, and Watson were the only actors to successfully pass the frequent pairs threshold, they formed the sole candidate combination capable of crossing the final support constraint.

7 Comments and Discussion

The empirical and qualitative findings collected across the analysis offer a comprehensive overview of the trade-offs between monolithic and partitioned frequent itemset mining architectures.

As for algorithmic correctness, all three pipelines were cross-referenced; at the baseline threshold, they completely converged on the exact same 79 singletons and perfectly reconstructed the tight collaborative cluster of the Harry Potter primary cast, confirming the mathematical consistency of the implementations.

These qualitative findings uncover an important domain-specific property of the dataset slice. While individual legendary actors appear across a diverse range of independent movies throughout their careers, crossing the singleton threshold but failing to form tight, repeating clusters, the strict contractual and narrative continuity of multi-installment franchises generates dense, deterministic itemsets. This explains why the combinatorial expansion at lower support thresholds is heavily driven by tightly-knit collaborative groups, providing a clear sociological and structural justification for the processing overhead observed during the global validation phases.

However, the computational paths taken to reach this identical output reveal a sharp architectural divergence under parametric stress. As long as the support constraint remains moderate ($R \geq 0.005$), all frameworks operate with negligible overhead, completing the mining process in less than 0.06 seconds. The monolithic approaches, A-Priori and PCY, particularly benefit from the compact nature of the chosen IMDb dataset slice: since the data fits entirely within the main memory (RAM), direct level-by-level validation is exceptionally fast, and PCY's bitmap filtering introduces virtually no tracking overhead, even when varying the hash table layout from 20 to 5,000 buckets.

The true behavioral inversion occurs at the lowest threshold ($R = 0.002$), where the dense co-occurrence web triggers an explosive growth to 787 frequent itemsets. Under these conditions, the SON framework suffers a severe performance degradation, with runtimes skyrocketing to ≈ 1.83 seconds, making it roughly 285 times slower than A-Priori.

This penalty is a direct consequence of the "candidate congestion" inherent to the data partitioning strategy when applied to low-density, small-scale datasets. As demonstrated by the mathematical threshold collapse examined in Section 4, splitting a small dataset under ultra-low global constraints forces the localized support requirement to its absolute minimum. Consequently, the Map phase fails to perform any meaningful structural pruning, forcing nearly every localized item combination into the global candidate pool. The core bottleneck then shifts entirely to the Reduce phase, where the sequential verification of this massive volume of false-positive pairs and triples across the entire dataset introduces severe processing overhead.

In conclusion, the experiments demonstrate that while the data partitioning strategy of the SON framework is an indispensable architectural pattern for handling massive, distributed, and disk-bound datasets that cannot fit into memory, it introduces severe synchronization and filtering overhead when applied to small, in-memory tables under ultra-low support thresholds. Conversely, for compact and highly interconnected datasets like this IMDb slice, standard monolithic pipelines like A-Priori and PCY remain structurally superior, proving that algorithmic efficiency is highly dependent on the harmony between dataset scale, partitioning granularity, and parameter tuning.